Settings

There are many settings in Cassiopeia that control how the framework works, and more settings will be added as the code is expanded.

Use `cass.apply_settings(...)` and pass in a `json` filename, a dictionary, or a `cassiopeia.Settings` object to set Cass's parameters. Cass will use its own default settings if you do not run `cass.apply_settings`.

The method `cass.get_default_config()` will return a dictionary that contains the default settings that Cass uses. You can call this method, modify the returned dictionary, then pass it to `cass.apply_settings` to overwrite the default settings.

The most important setting is your Riot API key. It can be set programmatically (which will override the value specified in the settings).

Each setting is explained below, and should be added as separate entries to your settings dictionary/json.

Globals

The `"default_region"` setting should be set to the string version of the region that the Riot API requires (in all caps), for example `"NA"` for North America. This can be set programmatically using `cass.set_default_region`.

The `"version_from_match"` variable determines which version of the static data for matches is loaded (this includes, for example, the items for each participant). Valid values are `"version"`, `"patch"`, and `"latest"`. If set to `"version"`, the static data for the match's version will be loaded correctly; however, this requires pulling the match data for all matches. If you only want to use match reference data (and will not pull the full data for every match), you should use either `"patch"` or `"latest"`. `"patch"` will make a reasonable attempt to get the match's correct version based on its creation date (which is provided in the match reference data); however, if you pull a summoner's full match history, you will pull many versions of the static data, which may take a long time. In addition, the patch dates / times may be slightly off and may depend on the region. For small applications that barely uses the static data, pulling multiple versions of the static data is likely overkill. If that is the case, you should set this variable to `"latest"`, in which case the static data for the most recent version will be used; this, however, could result in missing or incorrect data if parts of the static data are accessed that have changed from patch to patch. The default is to use the patch if the match hasn't yet been loaded, which is a nice compromise between ensuring you, the user, always have correct data while also preventing new users from pulling a massive amount of unnecessary match data. It's likely that the patch dates aren't perfect, so be aware of this and please report and inconsistencies.

Below is an example:

```
{
    ...,
    "global": {
        "version_from_match": "patch",
        "default_region": null
    }
    ...
}
```

Data Pipeline

This setting is extremely important and therefore has its own page (Data Pipeline). However, our defaults will likely work for you if you're just getting started.

Riot API

The Riot API variable is an attribute of the `pipeline` variable, but it has a variety of settings relevant to the Riot API.

The `"api_key"` should be set to your Riot API key. You can instead supply an environment variable name that contains your API key (this is recommended so that you can push your settings file to version control without revealing your API key). This variable can be set programmatically via `cass.set_riot_api_key`.

The `"limit_sharing"` variable specifies what fraction of your API key should be used for your server. This is useful when you have multiple servers that you want to split your API key over. The default (if not set) is `1.0`, and valid values are between `0.0` and `1.0`.

Request Handling

The `"request_error_handling"` variable specifies how errors returned by the Riot API should be handled. There are three options, each of which has its own set of parameters: `"throw"` simply causes the error returned by the Riot API to be thrown to you, the user; `"exponential_backoff"` will exponentially backoff; and `"retry_from_headers"` will attempt to use the `"retry-after"` header in the response to retry after the specified amount of time. The `429` error code can be handled differently depending on which type of rate limiting cause it. See the example below for the specific structure for these settings.

`"throw"` takes no arguments.

`"exponential_backoff"` takes three arguments: `initial_backoff` specifies the initial time to pause before making another request, `backoff_factor` specifies what to multiply the `initial_backoff` by for each subsequent failure, and `max_attempts` specifies the maximum number of calls to make before throwing the error.

`"retry_from_headers"` takes one argument: `max_attempts` specifies the maximum number of calls to make before throwing the error.

Below is an example, and these settings are the default if any value is not specified:

```
"RiotAPI": {
    "api_key": "RIOT_API_KEY",
    "limiting_share": 1.0,
    "request_error_handling": {
        "404": {
            "strategy": "throw"
        },
        "429": {
            "service": {
                "strategy": "exponential_backoff",
                "initial_backoff": 1.0,
                "backoff_factor": 2.0,
                "max_attempts": 4
```

```
            },
            "method": {
                "strategy": "retry_from_headers",
                "max_attempts": 5
            },
            "application": {
                "strategy": "retry_from_headers",
                "max_attempts": 5
            }
        },
        "500": {
            "strategy": "throw"
        },
        "503": {
            "strategy": "throw"
        },
        "timeout": {
            "strategy": "throw"
        }
    }
}
```

An alternative setting for `request_error_handling` is below, which will retry 50x errors:

```
"request_error_handling": {
    "404": {
        "strategy": "throw"
    },
    "429": {
        "service": {
            "strategy": "exponential_backoff",
            "initial_backoff": 1.0,
            "backoff_factor": 2.0,
            "max_attempts": 4
        },
        "method": {
            "strategy": "retry_from_headers",
            "max_attempts": 5
        },
        "application": {
            "strategy": "retry_from_headers",
            "max_attempts": 5
        }
    },
    "500": {
        "strategy": "exponential_backoff",
        "initial_backoff": 1.0,
        "backoff_factor": 2.0,
        "max_attempts": 4
    },
    "503": {
        "strategy": "exponential_backoff",
        "initial_backoff": 1.0,
        "backoff_factor": 2.0,
        "max_attempts": 4
    },
    "timeout": {
        "strategy": "throw"
    },
    "403": {
        "strategy": "throw"
    }
```

```
}
```

Logging

The `"logging"` section defines variables related to logging and print statements.

The `"print_calls"` variable should be set to `true` or `false` and determines whether http calls (e.g. to the Riot API or Data Dragon) are printed. Similarly, the `"print_riot_api_key"` variable will print your Riot API key if set to `true`.

`"core"` and `"default"` are two loggers that are currently implemented in Cass, and you can set the logging levels using these variables. Acceptable values are the logging levels for python's logging module (e.g. `"INFO"` and `"WARNING"`).

Example:

```
"logging": {
    "print_calls": true,
    "print_riot_api_key": false,
    "default": "WARNING",
    "core": "WARNING"
}
```

Plugins

The `"plugins"` section defines which plugins Cassiopeia will use. See plugins for specifics for each plugin.